

26th June GFG Daily POTD

Distinct Subsequences (Count Ways to Form a String)

Problem Statement

Given two strings:

- `s1` (source string)
- `s2` (target string)

Return the **number of distinct subsequences** of `s1` that are equal to `s2`.

A subsequence is obtained by deleting zero or more characters **without changing the order** of the remaining characters.

Example

```
s1 = "babgbag"
s2 = "bag"

Output = 5
```

The five subsequences are formed by choosing different occurrences of '`b`', '`a`', and '`g`'.

Step 1 : Don't Think About DP

Forget DP completely.

Think only about choices.

At every character in `s1`, there are only two possibilities.

```
Take it
Skip it
```

Nothing else.

This observation itself creates recursion.

Brute Force Idea

Suppose

```
s1 = abc
```

```
At 'a'
```

```
  a
 /  \
take skip
```

Every character creates two branches.

Eventually every subsequence is generated.

Now simply count those equal to **s2**.

Recursive State

We need to know

```
Current index in s1
Current subsequence built
```

State

```
solve(i, cur)
```

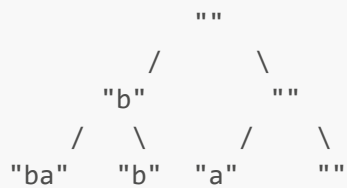
Meaning

```
i    -> current index in s1

cur  -> subsequence built till now
```

Recursive Tree

```
solve(0, "")
```



Eventually

```
cur == s2
```

means

Found one answer.

Brute Force Code

```
def countWaysBrute(s1, s2):
    ans = 0

    def solve(i, cur):
        nonlocal ans

        if i == len(s1):
            if cur == s2:
                ans += 1

            return

        # Pick
        solve(i + 1, cur + s1[i])

        # Skip
        solve(i + 1, cur)

    solve(0, "")

    return ans
```

Complexity

Every character has

```
Take  
Skip
```

Therefore

```
2 × 2 × 2 × ...  
  
= 2^n
```

Time

```
O(2^n)
```

Very slow.

Observation

Do we actually need to build the entire string?

Suppose

```
cur = "ba"
```

Why do we care about "ba"?

Actually we don't.

We only care that

```
First two characters of s2  
have already matched.
```

Instead of storing

```
"ba"
```

we can store

```
j = 2
```

because

```
Next character to match
```

```
s2[2]
```

This makes the state much smaller.

Better Recursive State

```
solve(i,j)
```

Meaning

```
Using s1[i:]
```

```
How many ways can I build
```

```
s2[j:] ?
```

Example

```
s1 = babgbag  
    ^
```

```
s2 = bag  
    ^
```

```
solve(0,0)
```

asks

```
Using entire s1,  
  
how many ways can I make  
entire s2?
```

Choices

Current character

```
s1[i]
```

Two possibilities.

Choice 1

Skip current character

```
solve(i+1,j)
```

Choice 2

If

```
s1[i]==s2[j]
```

then

Take it

```
solve(i+1,j+1)
```

Base Cases

Entire target matched

```
j==len(s2)
```

means

Already formed target.

Return

```
1
```

Source exhausted

```
i==len(s1)
```

Target still remaining.

Impossible.

Return

```
0
```

Final Recursive Code

```
def countWays(s1, s2):  
    def solve(i, j):  
        if j == len(s2):  
            return 1  
  
        if i == len(s1):  
            return 0  
  
        ans = solve(i + 1, j)  
  
        if s1[i] == s2[j]:  
            ans += solve(i + 1, j + 1)  
  
        return ans
```

```
return solve(0,0)
```

Why Memoization?

Imagine

```
solve(4,2)
```

is reached from

Path A

and

Path B

Without memoization

```
solve(4,2)
```

↓

Entire subtree computed

↓

Return

Later

```
solve(4,2)
```

↓

Entire subtree computed AGAIN

Huge waste.

Memoization Idea

Store answer for every

(i, j)

Once computed

Never compute again.

Memoization Code

```
from functools import lru_cache

def countWays(s1, s2):

    @lru_cache(None)
    def solve(i, j):

        if j == len(s2):
            return 1

        if i == len(s1):
            return 0

        ans = solve(i+1, j)

        if s1[i] == s2[j]:
            ans += solve(i+1, j+1)

        return ans

    return solve(0, 0)
```

Complexity

States

$(n+1)$

$\times$

```
(m+1)
```

Therefore

```
Time
```

```
O(nm)
```

```
Space
```

```
O(nm)
```

Now Comes DP

This is where most students memorize.

Instead, derive it.

Memoization State

```
solve(i,j)
```

means

```
Ways to build
```

```
s2[j:]
```

```
using
```

```
s1[i:]
```

Now simply rename

```
solve(i,j)
```

```
↓
```

```
dp[i][j]
```

Nothing changed.

DP state is literally recursion state.

DP Table

Suppose

```
n = len(s1)
```

```
m = len(s2)
```

Table

	j			
	0	1	2	3
i				
0				
1				
2				
3				
4				
5				
6				
7				

Each cell stores

```
Ways to form
```

```
s2[j:]
```

```
using  
  
s1[i:]
```

Base Case 1

Recursion

```
if j==m:  
    return 1
```

Therefore

Entire last column becomes

```
1
```

```
      0 1 2 3  
0      ? ? ? 1  
1      ? ? ? 1  
2      ? ? ? 1  
3      ? ? ? 1  
...  
n      ? ? ? 1
```

Reason

```
Empty target  
  
can always be formed.  
  
Just choose nothing.
```

Base Case 2

Recursion

```
if i==n:  
    return 0
```

Therefore

Last row

```
0 0 0 1
```

Why last one

```
1
```

because

```
Empty source  
  
can form  
  
Empty target.
```

Final initialization

```
      0 1 2 3  
0      ? ? ? 1  
1      ? ? ? 1  
2      ? ? ? 1  
...  
n      0 0 0 1
```

Transition

Recursion

```
Skip  
  
solve(i+1,j)
```

becomes

```
dp[i+1][j]
```

Take

```
solve(i+1,j+1)
```

becomes

```
dp[i+1][j+1]
```

Hence

```
dp[i][j]  
  
=  
  
dp[i+1][j]  
  
+  
  
dp[i+1][j+1]  
  
(if characters match)
```

Why Fill Bottom-Up?

Observe

```
dp[i][j]
depends on
dp[i+1][...]
```

The row below must already exist.

Therefore

```
Start
from bottom row
move upward.
```

Dry Run

Example

```
s1 = ab
s2 = a
```

Table

```

      a      ""
    0  1
a 0
b 1
""2
```

Initialize

```

      0  1
0    ?  1
```

1	?	1
2	0	1

Fill

```

i=1

'b'

!=

'a'

dp[1][0]

=

dp[2][0]

=

0

```

Now

```

i=0

'a'

==

'a'

dp[0][0]

=

dp[1][0]

+

dp[1][1]

=

0+1

```


=

1

Answer

1

Bottom-Up Code

```
def countWays(s1, s2):  
  
    n = len(s1)  
    m = len(s2)  
  
    dp = [[0]*(m+1) for _ in range(n+1)]  
  
    for i in range(n+1):  
        dp[i][m] = 1  
  
    for i in range(n-1,-1,-1):  
  
        for j in range(m-1,-1,-1):  
  
            dp[i][j] = dp[i+1][j]  
  
            if s1[i]==s2[j]:  
                dp[i][j] += dp[i+1][j+1]  
  
    return dp[0][0]
```

Visual Relation

Recursion

↓

solve(i,j)

↓

Memoization

```
↓  
  
Cache solve(i,j)  
  
↓  
  
DP  
  
↓  
  
dp[i][j]
```

Nothing changes except
how answers are stored.

General Pattern for String DP

Whenever you see

```
Two strings
```

Immediately think

```
Two pointers  
  
(i,j)
```

Common state

```
solve(i,j)
```

Common Choices

Skip

```
solve(i+1,j)
```

Match

```
solve(i+1,j+1)
```

Insert

```
solve(i,j+1)
```

Delete

```
solve(i+1,j)
```

Replace

```
solve(i+1,j+1)
```

Problems Using This Pattern

- Distinct Subsequences
 - Longest Common Subsequence (LCS)
 - Edit Distance
 - Wildcard Matching
 - Regular Expression Matching
 - Interleaving String
 - Minimum ASCII Delete Sum
 - Delete Operation for Two Strings
 - Shortest Common Supersequence
 - Sequence Alignment
-

General DP Derivation Template

Whenever solving a DP problem, follow this checklist.

Step 1

Define the recursive state.

What information completely describes my current position?

Step 2

Write the recursive choices.

Take
Skip
Insert
Delete
Match
Move

Step 3

Write base cases.

Ask

When should recursion stop?

Step 4

Write recursion.

Ignore optimization.

Step 5

Notice repeated states.

If

solve(i,j)

is called multiple times

↓

Memoization.

Step 6

Replace

```
solve(i,j)
```

↓

```
dp[i][j]
```

Step 7

Convert every recursive call

```
solve(...)
```

↓

```
dp[...]
```

Step 8

Determine filling order.

Rule:

```
If
```

```
dp[i]
```

```
depends on
```

```
dp[i+1]
```

↓

```
Fill from bottom.
```

```
-----
```

```
If  
  
dp[i]  
  
depends on  
  
dp[i-1]  
  
↓  
  
Fill from top.
```

```
-----  
  
If  
  
dp[j]  
  
depends on  
  
dp[j+1]  
  
↓  
  
Fill right to left.
```

```
-----  
  
If  
  
dp[j]  
  
depends on  
  
dp[j-1]  
  
↓  
  
Fill left to right.
```

Real-World Analogy

Imagine you're typing a document while comparing it with a reference document.

At every position you ask:

- Skip this character?
- Match it?
- Replace it?

- Delete it?

That's exactly how string DP works: each state represents "where am I in each document?" and each transition represents a possible action.

The Golden Rule of DP

Recursion defines the problem. Memoization avoids repeated work. DP is just memoization written as a table.

If you can derive the recursion yourself, the memoized and bottom-up DP versions usually follow naturally.